

CS0019 Modeling and Simulation: Midterm Reviewer

Dawn Andrei Pamesa | Modules 1 to 3 | Friday, October 9, 11 AM | Canvas

Use the definitions, contrasts, formulas, worked examples, and MATLAB syntax below for written review. Source: supplied M1, M2, and M3 slide text, picture-slide notes, and M3 slide PDF.

Module 1: Introduction to Modeling and Simulation

Core Concepts

Modeling and simulation (M&S) is a problem-based discipline for repeated hypothesis testing. Its four precepts are **modeling, simulation, visualization, and analysis**. A system is interacting elements with a common purpose; elements may include people, hardware, software, facilities, policies, and documents.

A **model** is a physical, mathematical, or logical representation of a system, entity, phenomenon, or process. **Simulation** operates or imitates a system through a model over time or space. **M&S** builds a model, executes it, and analyzes the resulting performance data for decisions. A system is the subject; a model represents it; simulation runs it.

Study a system through the actual system or a model. A model may be physical or mathematical; a mathematical model may yield an analytical solution or require simulation. Modelers quantify behavior, support design/control/use, and evaluate performance.

System Type	Meaning / Example
Physical / notional	Physical exists (factory line); notional is a concept for something not yet existing.
Discrete / continuous	Discrete state changes at separate instants (factory); continuous state changes smoothly over time (aircraft).
Open / closed	Open has exogenous activities from outside the boundary; closed has none.
Sampled-data	Underlying system is continuous, but observations are available only at discrete times.

System Components And Boundary

An **entity** is an object of interest; an **attribute** is its property; an **activity** is a process changing the system. The **state** collects relevant entity, attribute, and activity values at one time. The boundary determines what is internal. **Endogenous** activities occur inside it; **exogenous** activities originate outside it. **Deterministic** activity output is fully determined by input; **stochastic** activity output includes randomness.

System	Entity	Attribute	Activity
Supermarket	Customers	Shopping list	Checking out
Bank	Customers	Account number, balance, credit status	Deposits, withdrawals
Communication	Messages	Length, priority	Transmitting
Rapid rail	Passengers	Origin, destination	Traveling
Traffic	Cars	Speed, distance	Driving

System	Endogenous	Exogenous
Supermarket	Purchasing and checkout	Customer arrivals
Supermarket parking	Arrival of customers in supermarket	Arrival of cars

Memory Aid

Boundary test: ask whether an activity originates inside the chosen system. The same real-world event can be classified differently if the boundary changes.

Why And How To Model

Model when the system is inaccessible, dangerous or unacceptable to engage, or does not yet exist. First establish structure by setting the boundary and identifying entities, attributes, and activities. Then supply attribute values and define activity relationships.

Model type tree: physical models may be static or dynamic. Mathematical models may be static or dynamic; either may be addressed analytically or numerically, and dynamic numerical models lead to system simulation. A **static** model has no time evolution; a **dynamic** model changes with time.

Principles: block building represents interconnected input-output parts; relevance includes only what affects the question; accuracy uses correct information; aggregation groups individual entities into larger units.

Classification	First Case	Second Case
Static / dynamic	Snapshot without time progression	State evolves over time

Classification	First Case	Second Case
Deterministic / stochastic	Same input produces unique output	Random inputs or outputs
Continuous / discrete	State changes smoothly	State changes at events

Simulation History And Process

Timeline: 1940s Monte Carlo and neutron scattering; 1960s special-purpose languages such as SIMSCRIPT; 1970s mathematical foundations; 1980s PC software, GUIs, object orientation; 1990s web simulation, animation, optimization, and Markov-chain Monte Carlo.

Process: define the problem, identify important variables, construct the simulation model, set values to test, run it, examine results, and choose a course of action. Repeat when results suggest model changes.

Ten Steps In Simulation And Model Building

1. Define the goal.
2. Use a suitable mix of skills.
3. Involve the end user.
4. Choose suitable tools.
5. Set an appropriate level of detail.
6. Collect input data early.
7. Document the model.
8. Plan verification: does the implementation give the right answers for the model?
9. Plan validation: does the model answer the right questions about reality?
10. Analyze output statistically.

Development cycle: model (modeling technology) -> implement/code (development technology) -> execute simulation (computation) -> results -> analyze insight (data/information) -> revise model. Related disciplines: probability/statistics for random inputs and output, analysis/operations research, visualization, human factors, and project management.

Method	Best Fit	Result / Limitation
Analytical	Simple tractable model; algebra, calculus, probability	General or exact result under assumptions; may produce difficult integrals.

Method	Best Fit	Result / Limitation
Simulation	Complex or nonlinear model with delays, constraints, or many interactions	Numerical, scenario-specific estimates; requires runs, validation, and output analysis.

Strengths, Limits, And Paradigms

Advantages: explore system behavior without disturbing it; test changes and configurations; expose bottlenecks; diagnose interactions; compare policies. **Disadvantages:** model design needs expertise; data and development take time and money; stochastic results vary; interpreting results requires care.

Monte Carlo: repeatedly sample input distributions and compute output distributions.

Continuous simulation: state varies continuously in time, often through differential equations. **Discrete-event simulation (DES):** state changes at event times; clock advances to events and queuing models are common. Hybrid, real-time, and web-based simulation are other forms.

Fidelity is closeness of behavior to reality; **resolution** is degree of represented detail; **scale** is size of the represented scenario. A model can cover a large scale but omit fine detail. Slides compare chess, flight simulators, WoW, Halo, Doom, and WarSim along these dimensions.

Modeling methods: physics-based derives equations from physical laws; finite element divides a complex object into elements; data-based fits observed data; agent-based models interacting agents; aggregate combines small entities; hybrid combines paradigms. Other forms on the slides include Markov chains, finite-state automata, particle systems, queueing models, bond graphs, and Petri nets.

Simulation	Participants	Systems
Live	Real	Real
Virtual	Real	Simulated
Constructive	Simulated	Simulated

Applications: training, analysis, experimentation, engineering, and acquisition. Randomness distinguishes deterministic simulations (unique output for given input) from stochastic ones (random input and output). Domains named in the slides include military, transportation, decision support, games for learning, and medicine. Other examples include manufacturing, networks, computer systems, hospitals, inventory, and finance.

MATLAB Basics

MATLAB means Matrix Laboratory. Each variable is an array; `mynum = 6` is a 1 x 1 array. Names begin with a letter and then use letters, digits, or underscore; names are case-sensitive, length is limited by `namelengthmax`, and keywords are forbidden. Avoid shadowing built-in function names. `who` lists names, `whos` adds type/size details, `clear` clears all variables, and `clear x` clears `x`.

Class / Control	Rule / Example
Numeric	Default is double; single and signed/unsigned int8 through int64/uint64 exist. int8 range = -128 to 127; <code>intmin('int8')</code> , <code>intmax('int8')</code> .
Other classes	<code>char</code> for character arrays; logical for true/false. Cast with class functions.
Display	<code>format short</code> (default) or <code>long</code> changes displayed digits; <code>loose/compact</code> changes spacing; <code>2e4 = 20000</code> .
Special values	<code>pi</code> , <code>i/j</code> (imaginary unit), <code>Inf</code> , <code>NaN</code> . A user variable named <code>i</code> or <code>j</code> can overwrite the built-in meaning.
ASCII	<code>double('a') = 97</code> ; <code>char(97) = 'a'</code> ; <code>char('abcd'+1) = 'bcde'</code> .

Arithmetic: `+`, `-`, `*`, `/`, `\`, `^`; relational: `<`, `<=`, `>`, `>=`, `==`, `~=`; logical: `&&`, `||`, `~`, `xor`. MATLAB precedence follows parentheses, powers, unary negation, multiplication/division, addition/subtraction, relations, AND, OR, assignment. Use parentheses when uncertain. `fix`, `floor`, `ceil`, `round`, `mod`, `rem`, `sign`, `sqrt`, `nthroot`, `log` (natural), `log2`, `log10`, `exp`, `sin` (radians), and `sind` (degrees) are slide functions.

Vectors, Matrices, And Indexing

A scalar is 1 x 1; a column vector may be 3 x 1, a row vector 1 x 4, a matrix 2 x 3. `v = [1 2 3 4]` is a row vector; `1:5` gives `[1 2 3 4 5]`; `1:2:9` gives `[1 3 5 7 9]`. Semicolons separate rows: `[1;2;3]`. Apostrophe transposes a real row vector into a column. Matrix rows must have equal lengths.

MATLAB indices start at 1. `v(2)` selects element 2; `v(2:3)` selects a slice; `mat(2,3)` selects row 2, column 3; `mat(1:2,2:3)` selects a block. Assignment such as `b(2)=11` changes an element. `length(v)` gives vector length; for a matrix it gives its largest dimension. `[r,c]=size(mat)` gives row/column counts; `numel(mat)` gives total elements.

Command	Meaning / Slide Example
<code>reshape(mat,2,6)</code>	Reshapes a 3 x 4 matrix to 2 x 6, preserving columnwise element order.
<code>fliplr</code> / <code>flipud</code> / <code>rot90</code>	Reverse columns / reverse rows / rotate 90 degrees counterclockwise.
<code>repmat(intmat,3,2)</code>	Tiles a matrix 3 times vertically and 2 times horizontally.

Command	Meaning / Slide Example
<code>evect=[]</code>	Empty vector: length 0. <code>evect=[evect 4]</code> appends 4; <code>v(2)=[]</code> deletes vector element 2.
<code>min / sum / max / prod</code>	Reduce a vector; for a matrix operate on each column by default. <code>max(mat')</code> finds row maxima in a row vector; <code>max(mat,[],2)</code> returns column of row maxima.
<code>cumsum / cumprod</code>	<code>cumsum(1:5) = [1 3 6 10 15]</code> ; <code>cumprod(1:5) = [1 2 6 24 120]</code> .

Scalar versus array operations: `v*3` and `v/2` scale each element; `v1+v2` adds corresponding elements. For elementwise multiplication, division, and power use `.*`, `./`, and `.^`. Ordinary `*` performs matrix multiplication: A (2 x 3) times B (3 x 4) yields C (2 x 4). Slide output C = [35 46 17 19; 9 22 20 5]. `det([1 2 3; 2 3 4; 1 2 5]) = -2`.

Watch Out

The dot changes multiplication-based operators into elementwise operations. `A*B` requires inner dimensions to agree; `A.*B` requires compatible elementwise sizes. A row or column transpose changes dimensions.

Module 2: Discrete Event Simulation

Sets, Functions, And Probability

A **set** is a collection without duplicate elements; $\square$ is empty. A subset contains only elements of a larger set. Union includes elements in either set; intersection includes elements in both; difference removes the second set; complement includes elements in the sample space outside the set. Finite sets have a terminating count; countable sets can be paired with positive integers.

Worked set example: $A=\{2,4,10\}$, $B=\{4,6,8,10\}$, $S=\{2,4,6,8,10,12\}$. $A \sqcup B=\{2,4,6,8,10\}$; $A \cap B=\{4,10\}$; $A-B=\{2\}$; $B-A=\{6,8\}$; $A^c=\{6,8,12\}$.

A function maps each domain element to one codomain element. Its **range** is the outputs actually reached. It is **onto** if range = codomain and **one-to-one** if different inputs have different outputs. An inverse exists when both hold. For $f(x)=mx+b$ over real numbers, $m \neq 0$ gives inverse $f^{-1}(y)=(y-b)/m$; $m=0$ is constant and has no inverse.

A **probability space** (S,A,P) has a sample space S , a sigma algebra A of events, and a probability measure P . An event is a subset of S . A contains S and is closed under complements and countable unions (therefore intersections). For k outcomes, the full power set has 2^k events; the slide's finite example uses this full sigma algebra. P maps events to $[0,1]$, $P(S)=1$, and probabilities add for disjoint events.

Three-Sided Die Event	Probability
$\square$	0
$\{B\}$	1/6
$\{C\}$	1/3
$\{D\}$	1/2
$\{B,C\}$	1/2
$\{B,D\}$	2/3
$\{C,D\}$	5/6
$S=\{B,C,D\}$	1

A **random variable** X maps sample outcomes to real numbers, with $\{X \leq x\}$ an event for each x . Its **CDF** $F(x)=P(X \leq x)$, a nondecreasing step function for the die example. For a continuous X , the **PDF** $f(x)=dF/dx$ is nonnegative, integrates to 1, and $P(x_1 < X \leq x_2) = \int [x_1, x_2] f(x) dx$. A discrete variable has probability masses, not an ordinary continuous density.

Distribution	Parameters And Use
Uniform(a,b)	Minimum a, maximum b, $a < b$; values equally likely across a finite interval. Density $1/(b-a)$.

Distribution	Parameters And Use
Triangular(a,m,b)	Minimum a, most likely m, maximum b, $a < m < b$; use when approximate bounds and mode are known.
Exponential(m)	Mean interarrival time $m > 0$; rate $\lambda = 1/m$. Density $\lambda \exp(-\lambda x)$ for $x \geq 0$; $P(X > x) = \exp(-\lambda x)$.
Normal(m, σ)	Mean m, standard deviation $\sigma > 0$; variability around an average and sums of many influences.

Watch Out

For the exponential distribution, $\lambda \exp(-\lambda x)$ is the normalized density. The slide's displayed formula omits λ ; its survival probability $\exp(-\lambda x)$ is correct.

Random Variates And Data Models

A **random variate** is a sampled value from a chosen distribution. Generate a pseudorandom uniform number U in $[0,1)$, then transform it to the desired distribution. The linear congruential generator is $Z(k+1) = (aZ(k) + c) \bmod m$ and $U(k) = Z(k)/m$, where $Z(0)$ is the seed. The period is at most m ; a full cycle has period m .

Full-cycle conditions: c and m are relatively prime; every prime factor of m divides $a-1$; if 4 divides m , then 4 divides $a-1$. Slide example $a=1$, $c=3$, $m=20$, seed 2 gives Z : 2, 5, 8, 11, 14, 17, 0, ... and a full 20-value cycle because $\gcd(3,20)=1$ and $a-1=0$ satisfies the divisibility conditions.

Structural modeling defines entities, locations, resources, processes, and logic. **Data modeling** supplies quantitative values such as interarrival and service times, resource schedules, failures, and travel times. Simple input data may use samples, empirical distributions, or theoretical distributions; hard cases include no data, multimodal, correlated, or nonstationary data.

Input Choice	Advantage	Disadvantage
Direct sample	Only observed, legal values	Rare legal values may be missing; storage access and many draws can be slow.
Empirical distribution	Unlimited draws matching the observed sample	Cannot produce unobserved rare extremes.
Theoretical distribution	Smooths small-sample noise; efficient draws	Fitting parameters takes effort; fit may be poor.

Output data analysis turns simulation traces into performance measures. Estimate means and confidence intervals, and account for stochastic variation before comparing scenarios.

DES Mechanics

DES represents a dynamic system whose state changes instantaneously at separate event times. An **entity** moves through the system; an **attribute** records its individual data; a **resource** serves it with finite capacity; a **queue** holds it when service is unavailable; an **event** changes the state. Resource utilization = busy time / total observed time.

Five software features on the slides: entities, relationships, simulation executive, random-number generator, and results/statistics. The **simulation clock** is modeled time. The **future event list** orders scheduled events; state variables describe the current system; statistical accumulators collect performance data. Initialization sets the initial state; a timing routine chooses the next event; the event routine updates state; a random-variate library supplies draws; the main program coordinates; the report generator summarizes.

Flowchart: main program -> initialization -> timing (select next event) -> event routine (possibly draw from distribution library) -> termination test -> repeat timing if no, or report if yes. **Fixed-increment/time slicing** advances by constant Δt and may shift events to grid times. **Next-event** jumps to the earliest scheduled event and skips inactive time.

Memory Aid

At each DES event: advance the clock, update state, schedule any new events, and update counters. The clock does not tick through quiet periods under next-event advance.

Queueing Systems

Arrival variation and service-time variation create lines. Basic elements are customers/entities, server(s), and waiting queue(s). Interarrival time is the interval between arrivals; service time is time at the resource. A queue discipline determines who is served next: FIFO/FCFS, LIFO/LCFS, priority, SIRO (random order), or shortest processing time.

A customer may **jockey** by switching lines, **renege** by leaving after joining, **balk** by declining to join, or **faff** by delaying after checkout while gathering belongings. An analytical queue model is tractable for simpler systems; simulation handles complex multistation behavior. Balance service cost against customer waiting cost. The slide coffee-shop example minimizes total cost at **\$320 with two baristas**.

Key parameters: arrival rate, service rate, number of servers, optional maximum queue length, and optional source population. Four structures: single server/single phase (ATM), single server/multiple phases (Chipotle), multiple servers/single phase (shared line), and multiple servers/multiple phases (laundromat).

Watch Out

LIFO means Last In, First Out (slide says 'FIFO - Last in, First Out'). FIFO means First In, First Out.

Single-Server Formulas

For customer i : arrival A_i = prior arrival + interarrival; service begins $B_i = \max(A_i, E_{i-1})$; waiting $W_i = B_i - A_i$; service ends $E_i = B_i + S_i$; time in system $T_i = E_i - A_i = W_i + S_i$; server idle before i is $\max(0, A_i - E_{i-1})$. For the first arrival at time 0, $B_1 = 0$.

Watch Out

Waiting in queue = service begins – arrival (slide says 'arrival – service begins'). This must be nonnegative. Server idle time is also clamped at zero.

Customer	IAT	Arrival	Service	Begin	Wait	End	System	Idle
1	-	0	2	0	0	2	2	0
2	4	4	3	4	0	7	3	2
3	1	5	1	7	2	8	3	0
4	4	9	4	9	0	13	4	1
5	2	11	2	13	2	15	4	0
6	4	15	3	15	0	18	3	0
Total	15		15		4		19	3

Customer 3: $B_3 = \max(5, 7) = 7$, $W_3 = 7 - 5 = 2$, $E_3 = 7 + 1 = 8$, $T_3 = 8 - 5 = 3$. Over the six customers, average wait = $4/6$ minute and average system time = $19/6$ minutes. The server is idle 3 minutes before the last departure at $t=18$.

Module 3: Modeling Continuous Systems

Continuous-System Framework

A **continuous system** has state variables that evolve continuously with time. It can be represented as input $x(t) \rightarrow$ system $\rightarrow$ output $y(t)$. For a reservoir, the stored quantity q changes according to $dq/dt=x$ and output $y=q$. Examples on the slides span dams and tunnels, missile and aircraft systems, logistics, and business planning. Some logistics examples are naturally discrete-event and require a matching model choice.

Mathematical classes on the slides: first-order ODEs (predator-prey), second-order ODEs ($F=ma$, orbits, oscillators, ballistics, RLC circuits), and second-order PDEs (listed but not treated). Modeling strategy: understand the system's physics and turn it into quantitative equations; then choose a numerical solution method.

State variables collectively contain enough information to predict relevant future behavior at the required accuracy. Examples: position, velocity, mass, angle, current, voltage. **State equations** describe time derivatives of state. **Output equations** map state to quantities users need. The orbital example uses three-dimensional position and velocity as the state vector.

Predator-Prey Dynamics

The cycle: prey increase $\rightarrow$ predators have more food $\rightarrow$ predators increase $\rightarrow$ prey decline $\rightarrow$ predators starve $\rightarrow$ predators decline $\rightarrow$ prey recover. Let x be prey and y predators.

Kolmogorov form is $dx/dt=x f(x,y)$, $dy/dt=y g(x,y)$. The slide's Lotka-Volterra rates are $f=b-py$ and $g=rx-d$, so $dx/dt=x(b-py)$, $dy/dt=y(rx-d)$. Increasing predators lowers prey growth; increasing prey raises predator growth. With $b=r=d=p=1$, the slide plots closed orbits for different initial conditions.

Watch Out

The slide's printed conserved quantity for general b,p,r,d is inconsistent. From the displayed rate equations, a conserved expression is $r x - d \ln x + p y - b \ln y$. For $b=r=d=p=1$ this is $x - \ln x + y - \ln y$, up to sign and an additive constant.

Numerical Methods

Most coupled state equations have no simple closed-form solution. **Euler:** for $dx/dt=f(t,x)$, $x_{n+1} \approx x_n + h f(t_n, x_n)$. It follows the tangent slope over a short step h . **Midpoint Runge-Kutta (RK2) shown in the slides:** $t_{mid}=t_n+h/2$, $x_{mid}=x_n+(h/2)f(t_n, x_n)$, then $x_{n+1}=x_n+h f(t_{mid}, x_{mid})$. The midpoint slope usually improves on forward Euler for a similar step.

MATLAB Scripts, Functions, And Input/Output

Both scripts and functions use .m files. A **script** runs commands in the workspace with no input/output argument list. A **function** accepts arguments, returns output, and keeps internal variables local. Create or open a file with `edit filename.m`; the slides show making a directory, changing into it, and running a script. A radius-5 circle script computes $\text{area} = \pi \cdot 5^2 = 78.5398$.

Command / Code	Meaning
<code>input('Enter a number: ')</code>	Reads an evaluated numeric expression; assign it to a variable.
<code>input('Enter a string: ','s')</code>	Reads text literally. Empty Enter gives empty char input; leading spaces are retained.
<code>disp(x)</code>	Displays a value without its variable name.
<code>fprintf('...%d...\n',x)</code>	Formatted text; %d integer, %f fixed decimal, %e scientific, %g compact, %s string; \n newline and \t tab.
<code>fscanf(...)</code> / format	Formatted input / numeric display mode. format short, long, compact, loose control appearance.
<code>;</code>	Suppresses automatic command-window output.

Slide examples: `fprintf('%s will be %d this year.\n','Alice',12)` prints 'Alice will be 12 this year.' `disp(4^3)` displays 64. With `x=5` and `y=9`, the input script computes `z1=14`, `z2=-4`, `z3=45`, and `z4=7`. Use %f rather than %d for a potentially fractional average.

To print vectors, `fprintf('%d\n',vec)` writes one element per line; `fprintf('%d ',vec)` writes across a line. MATLAB feeds matrix elements to `fprintf` column by column, so choose format and transpose deliberately. Slide vector `[2 3 4 5]` appears as four lines with newline formatting or `2 3 4 5` with spaced formatting.

Plotting

Define `x` values, calculate `y=f(x)` elementwise, and call `plot(x,y)`. `xlabel`, `ylabel`, `title`, `grid on`, `axis equal`, `axis square`, `axis([xmin xmax ymin ymax])`, `hold on`, `legend`, `subplot`, and `figure` control presentation. The slide's plot commands also include `bar`, `loglog`, `semilogx`, `semilogy`, `stairs`, and `stem`.

LineSpec combines optional color, line style, and marker. Colors: `b` blue, `c` cyan, `g` green, `k` black, `m` magenta, `r` red, `w` white, `y` yellow. Styles: `-` solid, `--` dashed, `:` dotted, `-.` dash-dot. Markers include `o` circle, `+` plus, `*` star, `.` point, `x` cross, `s` square, `d` diamond, `^` upward triangle, and `p` pentagram. Example `plot(x,y,'r*')` shows red stars.

Slide plot examples: `plot(11,48,'r*')` marks one time/temperature point; `x=1:6`; `y=[1 5 3 9 11 8]`; `plot(x,y)` makes a line. `subplot(2,2,1)` selects the first tile of a 2 x 2 grid; `figure` opens

another window; `plot(x,y1,x,y2)` draws both curves. The sin/cos example sets x from 0 to 2π , $y1=\sin(x)$, $y2=\cos(x)$, then adds legend and labels. The final example plots $\exp(-1.5*x).*\sin(10*x)$ and $\exp(-2*x).*\sin(10*x)$ in two subplots.

Watch Out

Use `.*`, `./`, and `.^` in vector-valued function formulas. `axis equal` gives equal unit scale, while `axis square` gives a square plotting box.

User-Defined Functions

A file `calcarea.m` can contain `function area = calcarea(rad)`, then `area = pi * rad * rad; end`. Calling `calcarea(5)` yields about 78.54. The slide's `conevol.m` returns `outarg=(pi/3)*radius.^2.*height`; `conevol(4,6.1)` gives about 102.2065, and the displayed `conevol(3,5.5)` value is 51.84 to two decimals. Function name and file name must match for the primary function.

Selection And Loops

`if condition ... elseif condition ... else ... end` executes a matching branch. Slide example `a=100; if a<20 ... else fprintf('a is not less than 20\n');` end prints the else text. A sign test with `if x<0` prints Negative for negatives and Positive otherwise; zero belongs to the latter branch unless handled separately. The age example uses logical `&&` for ranges; ensure boundaries are exhaustive when adapting it.

Correct odd/even example: `x=input('Enter a number: '); if mod(x,2)==0; fprintf('Even number');` else; `fprintf('Odd number');` end. This is a valid MATLAB branch for integer x .

Watch Out

Use `mod(x,2)` and print Odd number in the else branch (slide says `x % 2` and prints 'Even number' in both branches). In MATLAB `%` starts a comment.

`switch grade; case 'A'; ...; case 'B'; ...; otherwise; ...; end` selects a matching case. With `grade='B'`, the slide prints 'Well done'. `menu('Pick a Pizza','Cheese','Sausage','Mushroom')` returns button number 1, 2, or 3; a switch then prints the chosen order. The slide's `x=y=3` switch prints 'x and y are equal'.

`for v=1:10; disp(v); end` prints 1 through 10. `for v=[1 5 8 17]` visits those four values; `for v=1.0:-0.2:0.0` counts down by 0.2; another loop prints Hello ten times. `while condition; statements; end` repeats while the condition remains true. The slides' example increments a from 10 to 19; another breaks when a reaches 15; another uses `continue` to skip displaying 15. Ensure the body eventually changes the condition or uses `break`.

One-Page Cram Sheet

M1 core: system = interacting parts; model = representation; simulation = run of model; M&S = build, execute, analyze. Four precepts: modeling, simulation, visualization, analysis. Entity/object; attribute/property; activity/change; state/snapshot. Open has exogenous activity; closed does not. Static/dynamic = no time/time; deterministic/stochastic = fixed/random; continuous/discrete = smooth/event changes.

M1 process: define goal and boundary -> model structure/data -> verify implementation -> validate model against purpose -> experiment -> statistically analyze output -> revise. Monte Carlo samples distributions; continuous uses differential equations; DES jumps between state-changing events. Fidelity = realism; resolution = detail; scale = scenario size. Live real/real; virtual real/simulated; constructive simulated/simulated.

MATLAB M1: indices start at 1; row [1 2 3], column [1;2;3], range 1:2:9. size dimensions, numel elements, length largest dimension. .* , ./ , .^ elementwise; * matrix product. det(A) determinant. who, whos, clear; format short/long; ASCII double('a')=97.

M2 probability: (S,A,P); full event set of k outcomes has 2^k subsets. CDF $F(x)=P(X\leq x)$; continuous PDF $f=dF/dx$, area=1. Uniform(bounds), triangular(bounds and mode), exponential(mean interarrival), normal(mean and SD). LCG $Z(k+1)=(aZ(k)+c) \bmod m$; $U=Z/m$. Full cycle: $\gcd(c,m)=1$; prime factors of m divide $a-1$; if $4|m$, $4|(a-1)$.

M2 queue: $B_i=\max(A_i, E_{i-1})$, $W_i=B_i-A_i$, $E_i=B_i+S_i$, $T_i=E_i-A_i$, $\text{idle}=\max(0, A_i-E_{i-1})$. Next-event advance jumps to earliest event. FIFO = first in, first out; LIFO = last in, first out. Six-customer totals: wait 4, time in system 19, idle 3 minutes.

M3 continuous: state equations determine derivatives, output equations calculate requested measurements. Lotka-Volterra: $dx/dt=x(b-py)$, $dy/dt=y(rx-d)$. Euler $x_{\text{next}}\approx x+h f(t,x)$; midpoint RK2 uses slope at $t+h/2$. MATLAB scripts use workspace; functions have local variables and return outputs. input(...,'s') reads text; fprintf uses %d, %f, %s and \n; plot(x,y,'r*'); if/elseif/else/end, switch/case/otherwise/end, for, while; odd/even uses mod(x,2).